

Project Folder Structure

```
perspective_api/
├── api/
│   ├── v1/
│   │   ├── endpoints/
│   │   │   ├── __init__.py
│   │   │   ├── column_state.py
│   │   │   ├── perspective.py
│   │   │   └── filter_model.py
│   │   └── __init__.py
│   └── __init__.py
├── database/
│   └── database.py
├── models/
│   └── perspective.py
├── schemas/
│   └── perspective.py
├── services/
│   └── perspective.py
└── main.py
```

main.py

```
import os
from flask import Flask, g
from api.v1.endpoints.perspective import perspective_bp
from api.v1.endpoints.column_state import column_state_bp
from api.v1.endpoints.filter_model import filter_model_bp
from database.database import close_db_connection

# Initialize the Flask application
app = Flask(__name__)

# Register the blueprints for the API routes
app.register_blueprint(perspective_bp, url_prefix='/api/v1')
app.register_blueprint(column_state_bp, url_prefix='/api/v1/column_state')
app.register_blueprint(filter_model_bp, url_prefix='/api/v1/filter_model')

# Add a teardown function to close the database connection and cursor
@app.teardown_appcontext
```

```

def teardown_db(exception=None):
    conn = g.pop('db_conn', None)
    curr = g.pop('db_curr', None)
    close_db_connection(conn, curr)

if __name__ == '__main__':
    app.run(debug=True)

```

api/v1/endpoints/perspective.py

```

from flask import Blueprint, request, jsonify, g
from pydantic import ValidationError
from ..schemas.perspective import Perspective, PerspectiveCreate, PerspectiveUpdate
from ..services.perspective import PerspectiveService
from ..database.database import get_db

perspective_bp = Blueprint('perspective', __name__)

@perspective_bp.route('/', methods=['GET'])
def get_all_perspectives_route():
    """
    Handles GET requests to retrieve all perspectives.
    """
    try:
        conn, curr = get_db()
        service = PerspectiveService(conn, curr)
        perspectives = service.get_all_perspectives()
        if not perspectives:
            return jsonify({"message": "No perspectives found"}), 404

        # Manually validate and serialize each item using Pydantic's from_attributes
        validated_perspectives = [Perspective.model_validate(p, from_attributes=True) for p in
perspectives]

        return jsonify([p.model_dump(mode='json') for p in validated_perspectives]), 200
    except ValidationError as e:
        return jsonify({"detail": e.errors()}), 400
    except Exception as e:
        return jsonify({"error": str(e)}), 500

```

```

@perspective_bp.route('/<int:perspective_id>', methods=['GET'])
def get_perspective_by_id_route(perspective_id):
    """
    Handles GET requests to retrieve a single perspective by its ID.
    """
    try:
        conn, curr = get_db()
        service = PerspectiveService(conn, curr)
        perspective = service.get_perspective_by_id(perspective_id)
        if not perspective:
            return jsonify({"message": f"Perspective with id {perspective_id} not found"}), 404

        # Validate and serialize the single item
        validated_perspective = Perspective.model_validate(perspective, from_attributes=True)
        return jsonify(validated_perspective.model_dump(mode='json')), 200
    except ValidationError as e:
        return jsonify({"detail": e.errors()}), 400
    except Exception as e:
        return jsonify({"error": str(e)}), 500

```

```

@perspective_bp.route('/user/<string:username>', methods=['GET'])
def get_perspective_by_username_route(username):
    """
    Handles GET requests to retrieve a single perspective by username.
    """
    try:
        conn, curr = get_db()
        service = PerspectiveService(conn, curr)
        perspective = service.get_perspective_by_username(username)
        if not perspective:
            return jsonify({"message": f"Perspective for user '{username}' not found"}), 404

        # Validate and serialize the single item
        validated_perspective = Perspective.model_validate(perspective, from_attributes=True)
        return jsonify(validated_perspective.model_dump(mode='json')), 200
    except ValidationError as e:
        return jsonify({"detail": e.errors()}), 400
    except Exception as e:
        return jsonify({"error": str(e)}), 500

```

```

@perspective_bp.route('/', methods=['POST'])
def create_perspective_route():
    """

```

Handles POST requests to create a new perspective.

"""

try:

data = request.json

perspective_in = PerspectiveCreate.model_validate(data)

conn, curr = get_db()

service = PerspectiveService(conn, curr)

new_perspective = service.create_perspective(perspective_in)

validated_new_perspective = Perspective.model_validate(new_perspective,
from_attributes=True)

return jsonify(validated_new_perspective.model_dump(mode='json')), 201

except ValidationError as e:

return jsonify({"detail": e.errors()}), 400

except Exception as e:

return jsonify({"error": str(e)}), 500

@perspective_bp.route('/<int:perspective_id>', methods=['PUT'])

def update_perspective_route(perspective_id):

"""

Handles PUT requests to update an existing perspective.

"""

try:

data = request.json

perspective_in = PerspectiveUpdate.model_validate(data)

conn, curr = get_db()

service = PerspectiveService(conn, curr)

updated_perspective = service.update_perspective(perspective_id, perspective_in)

if not updated_perspective:

return jsonify({"message": f"Perspective with id {perspective_id} not found"}), 404

validated_updated_perspective = Perspective.model_validate(updated_perspective,
from_attributes=True)

return jsonify(validated_updated_perspective.model_dump(mode='json')), 200

except ValidationError as e:

return jsonify({"detail": e.errors()}), 400

except Exception as e:

return jsonify({"error": str(e)}), 500

@perspective_bp.route('/<int:perspective_id>', methods=['DELETE'])

def delete_perspective_route(perspective_id):

"""

Handles DELETE requests to delete a perspective.

```

"""
try:
    conn, curr = get_db()
    service = PerspectiveService(conn, curr)
    deleted = service.delete_perspective(perspective_id)
    if not deleted:
        return jsonify({"message": f"Perspective with id {perspective_id} not found"}), 404
    return jsonify({"message": f"Perspective with id {perspective_id} deleted successfully"}),
200
except Exception as e:
    return jsonify({"error": str(e)}), 500

```

api/v1/endpoints/column_state.py

```

from flask import Blueprint, request, jsonify, g
from pydantic import ValidationError
from ..schemas.perspective import PerspectiveCreate, PerspectiveUpdate, ColumnState,
Perspective
from ..services.perspective import PerspectiveService
from ..database.database import get_db
from ..schemas.perspective import ViewSetting, FilterDetail

column_state_bp = Blueprint('column_state', __name__)

def _convert_view_settings_to_dicts(view_settings):
    """
    Recursively converts a list of ViewSetting DTOs to a list of dictionaries,
    including the nested FilterDetail DTOs.
    """
    if not view_settings:
        return []

    dict_list = []
    for vs in view_settings:
        filters_dict = {k: v.__dict__ for k, v in vs.filters.items()}
        dict_list.append({
            "name": vs.name,
            "view": vs.view,
            "filters": filters_dict,
            "default": vs.default

```

```
    })
    return dict_list
```

```
@column_state_bp.route('/save', methods=['POST'])
def save_column_state_route():
    """
    Handles POST requests to save a list of column_state items.
    If the user does not exist, a new perspective is created.
    If the user exists, the existing perspective's column_state is updated.
    """
    try:
        data = request.json
        username = data.get('username')
        column_state_data = data.get('column_state', [])

        if not username:
            return jsonify({"error": "Username is required."}), 400

        # Validate the incoming list of column_state data
        try:
            validated_column_state = [ColumnState.model_validate(cs) for cs in
column_state_data]
        except ValidationError as e:
            return jsonify({"error": "Invalid column_state data", "detail": e.errors()}), 400

        conn, curr = get_db()
        service = PerspectiveService(conn, curr)

        existing_perspective = service.get_perspective_by_username(username)

        if existing_perspective:
            # If perspective exists, update the column_state
            layout_name_from_body = data.get('layout_name')
            updated_by_from_body = data.get('updated_by')

            # Recursively convert nested DTOs to dictionaries for Pydantic validation
            sort_model_as_dict =
_convert_view_settings_to_dicts(existing_perspective.sort_model)
            filter_model_as_dict =
_convert_view_settings_to_dicts(existing_perspective.filter_model)

            perspective_update = PerspectiveUpdate(
```

```

        username=username,
        layout_name=layout_name_from_body if layout_name_from_body else
existing_perspective.layout_name,
        updated_by=updated_by_from_body if updated_by_from_body else
existing_perspective.updated_by,
        column_state=[cs.model_dump() for cs in validated_column_state],
        sort_model=sort_model_as_dict,
        filter_model=filter_model_as_dict
    )
    updated_perspective_model = service.update_perspective_by_username(username,
perspective_update)
    validated_updated_perspective =
Perspective.model_validate(updated_perspective_model, from_attributes=True)
    return jsonify(validated_updated_perspective.model_dump(mode='json')), 200
else:
    # If perspective does not exist, create a new one
    layout_name = data.get('layout_name')
    updated_by = data.get('updated_by')

    if not layout_name:
        return jsonify({"error": "layout_name is required for new perspectives."}), 400
    if not updated_by:
        return jsonify({"error": "updated_by is required for new perspectives."}), 400

    perspective_create = PerspectiveCreate(
        username=username,
        layout_name=layout_name,
        updated_by=updated_by,
        column_state=validated_column_state,
        sort_model=[],
        filter_model=[]
    )
    new_perspective_model = service.create_perspective(perspective_create)
    validated_new_perspective = Perspective.model_validate(new_perspective_model,
from_attributes=True)
    return jsonify(validated_new_perspective.model_dump(mode='json')), 201

except Exception as e:
    return jsonify({"error": str(e)}), 500

@column_state_bp.route('/save_single', methods=['POST'])
def save_single_column_state_route():
    """

```

Handles POST requests to save or update one or more column_state items.

This function implements an upsert logic based on the 'name' of each column state item.

- If a column_state with the same name exists, it is updated.
- If not, a new column_state item is added.
- If the user does not exist, a new perspective is created.

"""

try:

data = request.json

username = data.get('username')

column_state_data = data.get('column_state')

if not username:

return jsonify({"error": "Username is required."}), 400

if column_state_data is None:

return jsonify({"error": "column_state data is required."}), 400

column_states_to_process = column_state_data if isinstance(column_state_data, list) else
[column_state_data]

for item in column_states_to_process:

if not isinstance(item, dict) or 'name' not in item:

return jsonify({"error": "Each column state item must be a single JSON object with a
'name' field."}), 400

try:

validated_column_states = [ColumnState.model_validate(item) for item in
column_states_to_process]

except ValidationError as e:

return jsonify({"error": "Invalid column_state data", "detail": e.errors()}), 400

conn, curr = get_db()

service = PerspectiveService(conn, curr)

existing_perspective = service.get_perspective_by_username(username)

if existing_perspective:

User exists, so apply the upsert logic

existing_column_state_list = existing_perspective.column_state

for validated_item in validated_column_states:

If the incoming item is the new default, unset the old one

if validated_item.default:

```

        for existing_item in existing_column_state_list:
            existing_item.default = False

    # Convert the Pydantic model back to a dictionary for easier comparison and update
    validated_item_dict = validated_item.model_dump()
    # Ensure defaultColumns list has no duplicates
    validated_item_dict['defaultColumns'] =
list(set(validated_item_dict['defaultColumns']))

    found = False
    for i, existing_item in enumerate(existing_column_state_list):
        if existing_item.name == validated_item_dict['name']:
            # Update the existing DTO object's attributes
            existing_column_state_list[i].view = validated_item_dict['view']
            existing_column_state_list[i].defaultColumns =
validated_item_dict['defaultColumns']
            existing_column_state_list[i].default = validated_item_dict['default']
            found = True
            break

    if not found:
        # If no match was found, create a new DTO and append it to the list
        existing_column_state_list.append(ColumnState(**validated_item_dict))

    # Now, create a list of dictionaries from the updated list of DTOs for the Pydantic
model
    final_column_state_list_for_pydantic = [item.__dict__ for item in
existing_column_state_list]

    layout_name_from_body = data.get('layout_name')
    updated_by_from_body = data.get('updated_by')

    # Recursively convert nested DTOs to dictionaries for Pydantic validation
    sort_model_as_dict =
_convert_view_settings_to_dicts(existing_perspective.sort_model)
    filter_model_as_dict =
_convert_view_settings_to_dicts(existing_perspective.filter_model)

    perspective_update = PerspectiveUpdate(
        username=username,
        layout_name=layout_name_from_body if layout_name_from_body else
existing_perspective.layout_name,
        updated_by=updated_by_from_body if updated_by_from_body else

```

```

existing_perspective.updated_by,
    column_state=final_column_state_list_for_pydantic,
    sort_model=sort_model_as_dict,
    filter_model=filter_model_as_dict
)
    updated_perspective_model = service.update_perspective_by_username(username,
perspective_update)
    validated_updated_perspective =
Perspective.model_validate(updated_perspective_model, from_attributes=True)
    return jsonify(validated_updated_perspective.model_dump(mode='json')), 200
else:
    # User does not exist, create a new perspective
    layout_name = data.get('layout_name')
    updated_by = data.get('updated_by')

    if not layout_name:
        return jsonify({"error": "layout_name is required for new perspectives."}), 400
    if not updated_by:
        return jsonify({"error": "updated_by is required for new perspectives."}), 400

    # Use the list of validated items to create the new perspective
    perspective_create = PerspectiveCreate(
        username=username,
        layout_name=layout_name,
        updated_by=updated_by,
        column_state=validated_column_states,
        sort_model=[],
        filter_model=[]
    )
    new_perspective_model = service.create_perspective(perspective_create)
    validated_new_perspective = Perspective.model_validate(new_perspective_model,
from_attributes=True)
    return jsonify(validated_new_perspective.model_dump(mode='json')), 201

except Exception as e:
    return jsonify({"error": str(e)}), 500

@column_state_bp.route('/delete_single', methods=['DELETE'])
def delete_single_column_state_route():
    """
    Handles DELETE requests to remove a single column_state item by name.
    The request body must contain the username and the name of the column state to delete.
    """

```

```

try:
    data = request.json
    username = data.get('username')
    column_state_name_to_delete = data.get('column_state_name')

    if not username or not column_state_name_to_delete:
        return jsonify({"error": "Username and column_state_name are required."}), 400

    conn, curr = get_db()
    service = PerspectiveService(conn, curr)

    existing_perspective = service.get_perspective_by_username(username)

    if not existing_perspective:
        return jsonify({"message": f"Perspective for user '{username}' not found."}), 404

    # Step 1: Get the current list of column states from the existing perspective.
    original_column_state_list = existing_perspective.column_state

    # Step 2: Create a new list that excludes the item you want to delete.
    # This is a key part of the logic. We're filtering the list.
    updated_column_state_list = [
        cs for cs in original_column_state_list if cs.name != column_state_name_to_delete
    ]

    # Step 3: Check if the list size changed. If not, the item was not found.
    if len(updated_column_state_list) == len(original_column_state_list):
        return jsonify({"message": f"Column state with name '{column_state_name_to_delete}' not found."}), 404

    # Step 4: Convert the updated list of DTOs into a list of dictionaries.
    # This is necessary because the PerspectiveUpdate Pydantic model expects dictionaries.
    final_column_state_list_for_pydantic = [item.__dict__ for item in
updated_column_state_list]

    # Step 5: Prepare the PerspectiveUpdate object.
    # We need to correctly convert the nested DTOs in sort_model and filter_model to
dictionaries.
    sort_model_as_dict = _convert_view_settings_to_dicts(existing_perspective.sort_model)
    filter_model_as_dict = _convert_view_settings_to_dicts(existing_perspective.filter_model)

    perspective_update = PerspectiveUpdate(
        username=username,

```

```

        layout_name=existing_perspective.layout_name,
        updated_by=existing_perspective.updated_by,
        column_state=final_column_state_list_for_pydantic,
        sort_model=sort_model_as_dict,
        filter_model=filter_model_as_dict
    )

    # Step 6: Call the service layer to perform the database update.
    updated_perspective_model = service.update_perspective_by_username(username,
perspective_update)

    # Step 7: Validate the final model and return the JSON response.
    validated_updated_perspective =
Perspective.model_validate(updated_perspective_model, from_attributes=True)
    return jsonify(validated_updated_perspective.model_dump(mode='json')), 200

except Exception as e:
    return jsonify({"error": str(e)}), 500

```

api/v1/endpoints/filter_model.py

```

from flask import Blueprint, request, jsonify, g
from pydantic import ValidationError
from ..schemas.perspective import PerspectiveCreate, PerspectiveUpdate, Perspective,
ViewSetting, FilterDetail
from ..services.perspective import PerspectiveService
from ..database.database import get_db

filter_model_bp = Blueprint('filter_model', __name__)

def _convert_view_settings_to_dicts(view_settings):
    """
    Recursively converts a list of ViewSetting DTOs to a list of dictionaries,
    including the nested FilterDetail DTOs.
    """
    if not view_settings:
        return []

    dict_list = []
    for vs in view_settings:

```

```

        filters_dict = {k: v.__dict__ for k, v in vs.filters.items()}
        dict_list.append({
            "name": vs.name,
            "view": vs.view,
            "filters": filters_dict,
            "default": vs.default
        })
    return dict_list

@filter_model_bp.route('/save_single_filter', methods=['POST'])
def save_single_filter_model_route():
    """
    Handles POST requests to save or update one or more filter_model items.
    This function implements an upsert logic based on the combination of 'name' and 'view'.
    - If a filter_model item with the same name and view exists, it is updated.
    - If not, a new filter_model item is added.
    - If the user does not exist, a new perspective is created.
    """
    try:
        data = request.json
        username = data.get('username')
        filter_model_data = data.get('filter_model')

        if not username:
            return jsonify({"error": "Username is required."}), 400

        if filter_model_data is None:
            return jsonify({"error": "filter_model data is required."}), 400

        filters_to_process = filter_model_data if isinstance(filter_model_data, list) else
        [filter_model_data]

        # Validate the incoming list of filter_model data
        try:
            validated_filter_models = [ViewSetting.model_validate(fm) for fm in filters_to_process]
        except ValidationError as e:
            return jsonify({"error": "Invalid filter_model data", "detail": e.errors()}), 400

        conn, curr = get_db()
        service = PerspectiveService(conn, curr)

        existing_perspective = service.get_perspective_by_username(username)

```

```

if existing_perspective:
    # User exists, so apply the upsert logic
    existing_filter_model_list = existing_perspective.filter_model

    for validated_item in validated_filter_models:
        # Convert the Pydantic model back to a dictionary for easier comparison and update
        validated_item_dict = validated_item.model_dump()

        found = False
        for i, existing_item in enumerate(existing_filter_model_list):
            # Check for a match on both 'name' and 'view'
            if existing_item.name == validated_item_dict['name'] and existing_item.view ==
validated_item_dict['view']:
                # Update the existing DTO object's attributes
                existing_filter_model_list[i].filters = validated_item_dict['filters']
                existing_filter_model_list[i].default = validated_item_dict['default']
                found = True
                break

        if not found:
            # If no match was found, create a new DTO and append it to the list
            new_filters = {}
            for k, v in validated_item_dict.get('filters', {}).items():
                new_filters[k] = FilterDetail(**v)
            existing_filter_model_list.append(ViewSetting(
                name=validated_item_dict['name'],
                view=validated_item_dict['view'],
                filters=new_filters,
                default=validated_item_dict['default']
            ))

    # Now, create a list of dictionaries from the updated list of DTOs for the Pydantic
model
    final_filter_model_list_for_pydantic =
_convert_view_settings_to_dicts(existing_filter_model_list)

    layout_name_from_body = data.get('layout_name')
    updated_by_from_body = data.get('updated_by')

    # Convert existing nested DTOs to dictionaries for Pydantic validation
    sort_model_as_dict =
_convert_view_settings_to_dicts(existing_perspective.sort_model)
    column_state_as_dict = [cs.__dict__ for cs in existing_perspective.column_state]

```

```

        perspective_update = PerspectiveUpdate(
            username=username,
            layout_name=layout_name_from_body if layout_name_from_body else
existing_perspective.layout_name,
            updated_by=updated_by_from_body if updated_by_from_body else
existing_perspective.updated_by,
            column_state=column_state_as_dict,
            sort_model=sort_model_as_dict,
            filter_model=final_filter_model_list_for_pydantic
        )
        updated_perspective_model = service.update_perspective_by_username(username,
perspective_update)
        validated_updated_perspective =
Perspective.model_validate(updated_perspective_model, from_attributes=True)
        return jsonify(validated_updated_perspective.model_dump(mode='json')), 200
    else:
        # User does not exist, create a new perspective
        layout_name = data.get('layout_name')
        updated_by = data.get('updated_by')

        if not layout_name:
            return jsonify({"error": "layout_name is required for new perspectives."}), 400
        if not updated_by:
            return jsonify({"error": "updated_by is required for new perspectives."}), 400

        # Use the list of validated items to create the new perspective
        perspective_create = PerspectiveCreate(
            username=username,
            layout_name=layout_name,
            updated_by=updated_by,
            column_state=[],
            sort_model=[],
            filter_model=validated_filter_models
        )
        new_perspective_model = service.create_perspective(perspective_create)
        validated_new_perspective = Perspective.model_validate(new_perspective_model,
from_attributes=True)
        return jsonify(validated_new_perspective.model_dump(mode='json')), 201

    except Exception as e:
        return jsonify({"error": str(e)}), 500

```

database/database.py

```
import os
import psycopg2
from psycopg2.extras import DictCursor, register_uuid
from psycopg2.extensions import connection, cursor
from flask import g
from typing import Tuple

# Register UUID support for psycopg2
register_uuid()

# Database connection details (replace with your actual details or environment variables).
DB_NAME = "skg023"
DB_USER = "your_db_user"
DB_PASSWORD = "your_db_password"
DB_HOST = "localhost"
DB_PORT = "5432"
DB_SCHEMA = "recsui"

def get_db_connection() -> Tuple[connection, cursor]:
    """
    Establishes a connection to the PostgreSQL database and returns
    both the connection and a cursor object.

    Raises:
        psycopg2.Error: If the connection fails.
    """
    try:
        conn = psycopg2.connect(
            dbname=DB_NAME,
            user=DB_USER,
            password=DB_PASSWORD,
            host=DB_HOST,
            port=DB_PORT,
        )
        # Use DictCursor to return query results as dictionaries
        curr = conn.cursor(cursor_factory=DictCursor)
        curr.execute(f"SET search_path TO {DB_SCHEMA};")
        print("Successfully connected to the database.")
```

```

        return conn, curr
    except psycopg2.Error as e:
        print(f"Error connecting to the database: {e}")
        raise

def close_db_connection(conn: connection, curr: cursor):
    """Closes the database cursor and connection."""
    if curr:
        curr.close()
    if conn:
        conn.close()
    print("Database connection closed.")

def get_db():
    """
    Provides a database connection and cursor that is local to the current request.
    If they don't exist for the request, it creates new ones.
    """
    if 'db_conn' not in g or 'db_curr' not in g:
        try:
            conn, curr = get_db_connection()
            g.db_conn = conn
            g.db_curr = curr
        except psycopg2.Error as e:
            raise ConnectionError(f"Failed to get database connection: {e}") from e

    return g.db_conn, g.db_curr

```

models/perspective.py

```

import json
from typing import Optional, List, Dict, Any
from datetime import datetime

class ColumnState:
    """Represents the structure of a single item in the column_state array."""
    def __init__(self, name: str, view: str, defaultColumns: List[str], default: bool):
        self.name = name
        self.view = view
        self.defaultColumns = defaultColumns

```

```
self.default = default
```

```
class FilterDetail:
```

```
    """Represents the structure of a filter detail."""
```

```
    def __init__(self, type: str, filter: str):
```

```
        self.type = type
```

```
        self.filter = filter
```

```
class ViewSetting:
```

```
    """Represents the structure of a single item in the sort_model or filter_model arrays."""
```

```
    def __init__(self, name: str, view: str, filters: Dict[str, FilterDetail], default: bool):
```

```
        self.name = name
```

```
        self.view = view
```

```
        self.filters = filters
```

```
        self.default = default
```

```
class Perspective:
```

```
    """
```

```
    Data Transfer Object (DTO) for the 'perspectives' table.
```

```
    This class is not tied to an ORM and is used to represent a database row.
```

```
    """
```

```
    def __init__(self, id: int, username: str, layout_name: str, updated_by: str,
```

```
                  column_state: List[ColumnState], sort_model: List[ViewSetting],
```

```
                  filter_model: List[ViewSetting], updated_time: datetime):
```

```
        self.id = id
```

```
        self.username = username
```

```
        self.layout_name = layout_name
```

```
        self.updated_by = updated_by
```

```
        self.column_state = column_state
```

```
        self.sort_model = sort_model
```

```
        self.filter_model = filter_model
```

```
        self.updated_time = updated_time
```

```
@staticmethod
```

```
def from_dict(data: dict):
```

```
    """Converts a dictionary (from a psycopg2 query) to a Perspective object."""
```

```
    if not data:
```

```
        return None
```

```
# Manually deserialize JSONB data to handle potential type issues
```

```
try:
```

```
    column_state_data = json.loads(data.get('column_state', '[]'))
```

```
    column_state = [ColumnState(**cs) for cs in column_state_data]
```

```

except (json.JSONDecodeError, TypeError):
    column_state = []

try:
    sort_model_data = json.loads(data.get('sort_model', '[]'))
    sort_model = []
    for sm in sort_model_data:
        filters_dict = sm.get('filters', {})
        filters = {}
        if isinstance(filters_dict, dict):
            for k, v in filters_dict.items():
                if isinstance(k, str) and isinstance(v, dict):
                    filters[k] = FilterDetail(**v)
            sort_model.append(ViewSetting(name=sm['name'], view=sm['view'], filters=filters,
default=sm['default']))
    except (json.JSONDecodeError, TypeError):
        sort_model = []

try:
    filter_model_data = json.loads(data.get('filter_model', '[]'))
    filter_model = []
    for fm in filter_model_data:
        filters_dict = fm.get('filters', {})
        filters = {}
        if isinstance(filters_dict, dict):
            for k, v in filters_dict.items():
                if isinstance(k, str) and isinstance(v, dict):
                    filters[k] = FilterDetail(**v)
            filter_model.append(ViewSetting(name=fm['name'], view=fm['view'], filters=filters,
default=fm['default']))
    except (json.JSONDecodeError, TypeError):
        filter_model = []

return Perspective(
    id=data['id'],
    username=data['username'],
    layout_name=data['layout_name'],
    updated_by=data['updated_by'],
    column_state=column_state,
    sort_model=sort_model,
    filter_model=filter_model,
    updated_time=data['updated_time']
)

```

schemas/perspective.py

```
from datetime import datetime
from typing import Optional, List, Dict, Any, Union
from pydantic import BaseModel, Field, model_serializer, validator

# Schema for the nested filter details within sort and filter models
class FilterDetail(BaseModel):
    type: str
    filter: str

# Schema for the column_state data structure
class ColumnState(BaseModel):
    name: str
    view: str
    defaultColumns: List[str]
    default: bool

# Schema for the sort_model and filter_model data structures
class ViewSetting(BaseModel):
    name: str
    view: str
    filters: Dict[str, FilterDetail]
    default: bool

    @validator('filters', pre=True)
    def validate_filters(cls, v):
        """
        Validator to convert an empty string for `filters` to an empty dictionary.
        This handles cases where the incoming JSON sends "filters": "".
        """
        if isinstance(v, str) and v == "":
            return {}
        return v

# Base schema for creating or updating a perspective.
# All fields are validated to be non-empty strings as requested.
class PerspectiveBase(BaseModel):
    username: str = Field(..., min_length=1, description="Username cannot be empty")
```

```

layout_name: str = Field(..., min_length=1, description="Layout name cannot be empty")
updated_by: str = Field(..., min_length=1, description="Updater's email cannot be empty")
column_state: Optional[List[ColumnState]] = []
sort_model: Optional[List[ViewSetting]] = []
filter_model: Optional[List[ViewSetting]] = []

# Schema for creating a new perspective (inherits from PerspectiveBase)
class PerspectiveCreate(PerspectiveBase):
    pass

# Schema for updating an existing perspective.
# All fields are optional for partial updates.
class PerspectiveUpdate(PerspectiveBase):
    username: Optional[str] = Field(None, min_length=1, description="Username cannot be empty")
    layout_name: Optional[str] = Field(None, min_length=1, description="Layout name cannot be empty")
    updated_by: Optional[str] = Field(None, min_length=1, description="Updater's email cannot be empty")
    column_state: Optional[List[ColumnState]] = None
    sort_model: Optional[List[ViewSetting]] = None
    filter_model: Optional[List[ViewSetting]] = None

# Full schema for a perspective, including auto-generated fields.
class Perspective(PerspectiveBase):
    id: int
    updated_time: datetime

    class Config:
        from_attributes = True

```

services/perspective.py

```

import psycopg2
from psycopg2.extras import DictCursor
from typing import List, Optional
import json
from ..models.perspective import Perspective as PerspectiveModel
from ..schemas.perspective import PerspectiveCreate, PerspectiveUpdate
from psycopg2.extensions import connection, cursor

```

```

class PerspectiveService:
    """Service class for performing CRUD operations on Perspective data using psycopg2."""

    def __init__(self, db_conn: connection, db_curr: cursor):
        self.db_conn = db_conn
        self.db_curr = db_curr

    def get_all_perspectives(self) -> List[PerspectiveModel]:
        """Retrieves all perspective records from the database."""
        self.db_curr.execute("SELECT * FROM recsui.perspectives;")
        perspectives = self.db_curr.fetchall()
        return [PerspectiveModel.from_dict(p) for p in perspectives]

    def get_perspective_by_id(self, perspective_id: int) -> Optional[PerspectiveModel]:
        """Retrieves a single perspective record by its ID."""
        self.db_curr.execute("SELECT * FROM recsui.perspectives WHERE id = %s;",
        (perspective_id,))
        perspective = self.db_curr.fetchone()
        return PerspectiveModel.from_dict(perspective)

    def get_perspective_by_username(self, username: str) -> Optional[PerspectiveModel]:
        """Retrieves a single perspective record by its username."""
        self.db_curr.execute("SELECT * FROM recsui.perspectives WHERE username = %s;",
        (username,))
        perspective = self.db_curr.fetchone()
        return PerspectiveModel.from_dict(perspective)

    def create_perspective(self, perspective_in: PerspectiveCreate) -> PerspectiveModel:
        """Creates a new perspective record in the database."""
        # Convert Pydantic models to JSON strings for database insertion
        column_state_json = json.dumps([cs.model_dump() for cs in
        perspective_in.column_state])
        sort_model_json = json.dumps([sm.model_dump() for sm in perspective_in.sort_model])
        filter_model_json = json.dumps([fm.model_dump() for fm in perspective_in.filter_model])

        try:
            self.db_curr.execute(
                """
                INSERT INTO recsui.perspectives (username, layout_name, updated_by,
        column_state, sort_model, filter_model)
                VALUES (%s, %s, %s, %s, %s, %s)
                RETURNING *;
            """

```

```

        """
        (
            perspective_in.username,
            perspective_in.layout_name,
            perspective_in.updated_by,
            column_state_json,
            sort_model_json,
            filter_model_json
        )
    )
    new_perspective = self.db_curr.fetchone()
    self.db_conn.commit()
    return PerspectiveModel.from_dict(new_perspective)
except Exception as e:
    self.db_conn.rollback()
    raise e

```

def update_perspective(self, perspective_id: int, perspective_in: PerspectiveUpdate) -> Optional[PerspectiveModel]:

```

    """Updates an existing perspective record."""
    # Check if the perspective exists
    perspective_to_update = self.get_perspective_by_id(perspective_id)
    if not perspective_to_update:
        return None

    # Build the update query dynamically
    update_clauses = []
    update_data = []

    # Convert Pydantic models to JSON strings for database update
    perspective_dict = perspective_in.model_dump(exclude_unset=True)

    if 'username' in perspective_dict:
        update_clauses.append("username = %s")
        update_data.append(perspective_dict['username'])
    if 'layout_name' in perspective_dict:
        update_clauses.append("layout_name = %s")
        update_data.append(perspective_dict['layout_name'])
    if 'updated_by' in perspective_dict:
        update_clauses.append("updated_by = %s")
        update_data.append(perspective_dict['updated_by'])
    if 'column_state' in perspective_dict:
        update_clauses.append("column_state = %s")

```

```

        update_data.append(json.dumps([cs.model_dump() for cs in
perspective_in.column_state]))
        if 'sort_model' in perspective_dict:
            update_clauses.append("sort_model = %s")
            update_data.append(json.dumps([sm.model_dump() for sm in
perspective_in.sort_model]))
        if 'filter_model' in perspective_dict:
            update_clauses.append("filter_model = %s")
            update_data.append(json.dumps([fm.model_dump() for fm in
perspective_in.filter_model]))

    if not update_clauses:
        return perspective_to_update # No changes to apply

    query = f"UPDATE recsui.perspectives SET {' '.join(update_clauses)} WHERE id = %s
RETURNING *;"
    update_data.append(perspective_id)

    try:
        self.db_curr.execute(query, tuple(update_data))
        updated_perspective = self.db_curr.fetchone()
        self.db_conn.commit()
        return PerspectiveModel.from_dict(updated_perspective)
    except Exception as e:
        self.db_conn.rollback()
        raise e

def update_perspective_by_username(self, username: str, perspective_in:
PerspectiveUpdate) -> Optional[PerspectiveModel]:
    """Updates an existing perspective record by its username."""
    # Check if the perspective exists
    perspective_to_update = self.get_perspective_by_username(username)
    if not perspective_to_update:
        return None

    # Build the update query dynamically
    update_clauses = []
    update_data = []

    # Convert Pydantic models to JSON strings for database update
    perspective_dict = perspective_in.model_dump(exclude_unset=True)

    if 'username' in perspective_dict:

```

```

        update_clauses.append("username = %s")
        update_data.append(perspective_dict['username'])
    if 'layout_name' in perspective_dict:
        update_clauses.append("layout_name = %s")
        update_data.append(perspective_dict['layout_name'])
    if 'updated_by' in perspective_dict:
        update_clauses.append("updated_by = %s")
        update_data.append(perspective_dict['updated_by'])
    if 'column_state' in perspective_dict:
        update_clauses.append("column_state = %s")
        update_data.append(json.dumps([cs.model_dump() for cs in
perspective_in.column_state]))
    if 'sort_model' in perspective_dict:
        update_clauses.append("sort_model = %s")
        update_data.append(json.dumps([sm.model_dump() for sm in
perspective_in.sort_model]))
    if 'filter_model' in perspective_dict:
        update_clauses.append("filter_model = %s")
        update_data.append(json.dumps([fm.model_dump() for fm in
perspective_in.filter_model]))

    if not update_clauses:
        return PerspectiveModel.from_dict(perspective_to_update) # No changes to apply

    query = f"UPDATE recsui.perspectives SET {' '.join(update_clauses)} WHERE username =
%s RETURNING *;"
    update_data.append(username)

    try:
        self.db_curr.execute(query, tuple(update_data))
        updated_perspective = self.db_curr.fetchone()
        self.db_conn.commit()
        return PerspectiveModel.from_dict(updated_perspective)
    except Exception as e:
        self.db_conn.rollback()
        raise e

def delete_perspective(self, perspective_id: int) -> bool:
    """Deletes a perspective record by its ID."""
    try:
        self.db_curr.execute("DELETE FROM recsui.perspectives WHERE id = %s RETURNING
id;", (perspective_id,))

```

```

        deleted_row = self.db_curr.fetchone()
        if deleted_row:
            self.db_conn.commit()
            return True
        else:
            self.db_conn.rollback()
            return False
    except Exception as e:
        self.db_conn.rollback()
        raise e

```

api/v1/endpoints/filter_model.py

```

from flask import Blueprint, request, jsonify, g
from pydantic import ValidationError
from ..schemas.perspective import PerspectiveCreate, PerspectiveUpdate, Perspective,
ViewSetting, FilterDetail
from ..services.perspective import PerspectiveService
from ..database.database import get_db

filter_model_bp = Blueprint('filter_model', __name__)

def _convert_view_settings_to_dicts(view_settings):
    """
    Recursively converts a list of ViewSetting DTOs to a list of dictionaries,
    including the nested FilterDetail DTOs.
    """
    if not view_settings:
        return []

    dict_list = []
    for vs in view_settings:
        filters_dict = {k: v.__dict__ for k, v in vs.filters.items()}
        dict_list.append({
            "name": vs.name,
            "view": vs.view,
            "filters": filters_dict,
            "default": vs.default
        })
    return dict_list

```

```

@filter_model_bp.route('/save_single_filter', methods=['POST'])
def save_single_filter_model_route():
    """
    Handles POST requests to save or update one or more filter_model items.
    This function implements an upsert logic based on the combination of 'name' and 'view'.
    - If a filter_model item with the same name and view exists, it is updated.
    - If not, a new filter_model item is added.
    - If the user does not exist, a new perspective is created.
    """
    try:
        data = request.json
        username = data.get('username')
        filter_model_data = data.get('filter_model')

        if not username:
            return jsonify({"error": "Username is required."}), 400

        if filter_model_data is None:
            return jsonify({"error": "filter_model data is required."}), 400

        filters_to_process = filter_model_data if isinstance(filter_model_data, list) else
[filter_model_data]

        # Validate the incoming list of filter_model data
        try:
            validated_filter_models = [ViewSetting.model_validate(fm) for fm in filters_to_process]
        except ValidationError as e:
            return jsonify({"error": "Invalid filter_model data", "detail": e.errors()}), 400

        conn, curr = get_db()
        service = PerspectiveService(conn, curr)

        existing_perspective = service.get_perspective_by_username(username)

        if existing_perspective:
            # User exists, so apply the upsert logic
            existing_filter_model_list = existing_perspective.filter_model

            for validated_item in validated_filter_models:
                # Convert the Pydantic model back to a dictionary for easier comparison and update
                validated_item_dict = validated_item.model_dump()

```

```

found = False
for i, existing_item in enumerate(existing_filter_model_list):
    # Check for a match on both 'name' and 'view'
    if existing_item.name == validated_item_dict['name'] and existing_item.view ==
validated_item_dict['view']:
        # Update the existing DTO object's attributes
        existing_filter_model_list[i].filters = validated_item_dict['filters']
        existing_filter_model_list[i].default = validated_item_dict['default']
        found = True
        break

if not found:
    # If no match was found, create a new DTO and append it to the list
    new_filters = {}
    for k, v in validated_item_dict.get('filters', {}).items():
        new_filters[k] = FilterDetail(**v)
    existing_filter_model_list.append(ViewSetting(
        name=validated_item_dict['name'],
        view=validated_item_dict['view'],
        filters=new_filters,
        default=validated_item_dict['default']
    ))

# Now, create a list of dictionaries from the updated list of DTOs for the Pydantic
model
final_filter_model_list_for_pydantic =
_convert_view_settings_to_dicts(existing_filter_model_list)

layout_name_from_body = data.get('layout_name')
updated_by_from_body = data.get('updated_by')

# Convert existing nested DTOs to dictionaries for Pydantic validation
sort_model_as_dict =
_convert_view_settings_to_dicts(existing_perspective.sort_model)
column_state_as_dict = [cs.__dict__ for cs in existing_perspective.column_state]

perspective_update = PerspectiveUpdate(
    username=username,
    layout_name=layout_name_from_body if layout_name_from_body else
existing_perspective.layout_name,
    updated_by=updated_by_from_body if updated_by_from_body else
existing_perspective.updated_by,
    column_state=column_state_as_dict,

```

```

        sort_model=sort_model_as_dict,
        filter_model=final_filter_model_list_for_pydantic
    )
    updated_perspective_model = service.update_perspective_by_username(username,
perspective_update)
    validated_updated_perspective =
Perspective.model_validate(updated_perspective_model, from_attributes=True)
    return jsonify(validated_updated_perspective.model_dump(mode='json')), 200
else:
    # User does not exist, create a new perspective
    layout_name = data.get('layout_name')
    updated_by = data.get('updated_by')

    if not layout_name:
        return jsonify({"error": "layout_name is required for new perspectives."}), 400
    if not updated_by:
        return jsonify({"error": "updated_by is required for new perspectives."}), 400

    # Use the list of validated items to create the new perspective
    perspective_create = PerspectiveCreate(
        username=username,
        layout_name=layout_name,
        updated_by=updated_by,
        column_state=[],
        sort_model=[],
        filter_model=validated_filter_models
    )
    new_perspective_model = service.create_perspective(perspective_create)
    validated_new_perspective = Perspective.model_validate(new_perspective_model,
from_attributes=True)
    return jsonify(validated_new_perspective.model_dump(mode='json')), 201

except Exception as e:
    return jsonify({"error": str(e)}), 500

```

database/database.py

```

import os
import psycpg2
from psycpg2.extras import DictCursor, register_uuid

```

```

from psycopg2.extensions import connection, cursor
from flask import g
from typing import Tuple

# Register UUID support for psycopg2
register_uuid()

# Database connection details (replace with your actual details or environment variables).
DB_NAME = "skg023"
DB_USER = "your_db_user"
DB_PASSWORD = "your_db_password"
DB_HOST = "localhost"
DB_PORT = "5432"
DB_SCHEMA = "recsui"

def get_db_connection() -> Tuple[connection, cursor]:
    """
    Establishes a connection to the PostgreSQL database and returns
    both the connection and a cursor object.

    Raises:
        psycopg2.Error: If the connection fails.
    """
    try:
        conn = psycopg2.connect(
            dbname=DB_NAME,
            user=DB_USER,
            password=DB_PASSWORD,
            host=DB_HOST,
            port=DB_PORT,
        )
        # Use DictCursor to return query results as dictionaries
        curr = conn.cursor(cursor_factory=DictCursor)
        curr.execute(f"SET search_path TO {DB_SCHEMA};")
        print("Successfully connected to the database.")
        return conn, curr
    except psycopg2.Error as e:
        print(f"Error connecting to the database: {e}")
        raise

def close_db_connection(conn: connection, curr: cursor):
    """Closes the database cursor and connection."""
    if curr:

```

```

        curr.close()
    if conn:
        conn.close()
    print("Database connection closed.")

def get_db():
    """
    Provides a database connection and cursor that is local to the current request.
    If they don't exist for the request, it creates new ones.
    """
    if 'db_conn' not in g or 'db_curr' not in g:
        try:
            conn, curr = get_db_connection()
            g.db_conn = conn
            g.db_curr = curr
        except psycopg2.Error as e:
            raise ConnectionError(f"Failed to get database connection: {e}") from e

    return g.db_conn, g.db_curr

```

models/perspective.py

```

import json
from typing import Optional, List, Dict, Any
from datetime import datetime

class ColumnState:
    """Represents the structure of a single item in the column_state array."""
    def __init__(self, name: str, view: str, defaultColumns: List[str], default: bool):
        self.name = name
        self.view = view
        self.defaultColumns = defaultColumns
        self.default = default

class FilterDetail:
    """Represents the structure of a filter detail."""
    def __init__(self, type: str, filter: str):
        self.type = type
        self.filter = filter

```

```

class ViewSetting:
    """Represents the structure of a single item in the sort_model or filter_model arrays."""
    def __init__(self, name: str, view: str, filters: Dict[str, FilterDetail], default: bool):
        self.name = name
        self.view = view
        self.filters = filters
        self.default = default

class Perspective:
    """
    Data Transfer Object (DTO) for the 'perspectives' table.
    This class is not tied to an ORM and is used to represent a database row.
    """
    def __init__(self, id: int, username: str, layout_name: str, updated_by: str,
                  column_state: List[ColumnState], sort_model: List[ViewSetting],
                  filter_model: List[ViewSetting], updated_time: datetime):
        self.id = id
        self.username = username
        self.layout_name = layout_name
        self.updated_by = updated_by
        self.column_state = column_state
        self.sort_model = sort_model
        self.filter_model = filter_model
        self.updated_time = updated_time

    @staticmethod
    def from_dict(data: dict):
        """Converts a dictionary (from a psycopg2 query) to a Perspective object."""
        if not data:
            return None

        # Manually deserialize JSONB data to handle potential type issues
        try:
            column_state_data = json.loads(data.get('column_state', '[]'))
            column_state = [ColumnState(**cs) for cs in column_state_data]
        except (json.JSONDecodeError, TypeError):
            column_state = []

        try:
            sort_model_data = json.loads(data.get('sort_model', '[]'))
            sort_model = []
            for sm in sort_model_data:
                filters_dict = sm.get('filters', {})

```

```

        filters = {}
        if isinstance(filters_dict, dict):
            for k, v in filters_dict.items():
                if isinstance(k, str) and isinstance(v, dict):
                    filters[k] = FilterDetail(**v)
            sort_model.append(ViewSetting(name=sm['name'], view=sm['view'], filters=filters,
default=sm['default']))
        except (json.JSONDecodeError, TypeError):
            sort_model = []

    try:
        filter_model_data = json.loads(data.get('filter_model', '[]'))
        filter_model = []
        for fm in filter_model_data:
            filters_dict = fm.get('filters', {})
            filters = {}
            if isinstance(filters_dict, dict):
                for k, v in filters_dict.items():
                    if isinstance(k, str) and isinstance(v, dict):
                        filters[k] = FilterDetail(**v)
            filter_model.append(ViewSetting(name=fm['name'], view=fm['view'], filters=filters,
default=fm['default']))
        except (json.JSONDecodeError, TypeError):
            filter_model = []

    return Perspective(
        id=data['id'],
        username=data['username'],
        layout_name=data['layout_name'],
        updated_by=data['updated_by'],
        column_state=column_state,
        sort_model=sort_model,
        filter_model=filter_model,
        updated_time=data['updated_time']
    )

```

schemas/perspective.py

```

from datetime import datetime
from typing import Optional, List, Dict, Any, Union

```

```

from pydantic import BaseModel, Field, model_serializer, validator

# Schema for the nested filter details within sort and filter models
class FilterDetail(BaseModel):
    type: str
    filter: str

# Schema for the column_state data structure
class ColumnState(BaseModel):
    name: str
    view: str
    defaultColumns: List[str]
    default: bool

# Schema for the sort_model and filter_model data structures
class ViewSetting(BaseModel):
    name: str
    view: str
    filters: Dict[str, FilterDetail]
    default: bool

    @validator('filters', pre=True)
    def validate_filters(cls, v):
        """
        Validator to convert an empty string for `filters` to an empty dictionary.
        This handles cases where the incoming JSON sends "filters": "".
        """
        if isinstance(v, str) and v == "":
            return {}
        return v

# Base schema for creating or updating a perspective.
# All fields are validated to be non-empty strings as requested.
class PerspectiveBase(BaseModel):
    username: str = Field(..., min_length=1, description="Username cannot be empty")
    layout_name: str = Field(..., min_length=1, description="Layout name cannot be empty")
    updated_by: str = Field(..., min_length=1, description="Updater's email cannot be empty")
    column_state: Optional[List[ColumnState]] = []
    sort_model: Optional[List[ViewSetting]] = []
    filter_model: Optional[List[ViewSetting]] = []

# Schema for creating a new perspective (inherits from PerspectiveBase)
class PerspectiveCreate(PerspectiveBase):

```

```
pass
```

```
# Schema for updating an existing perspective.
```

```
# All fields are optional for partial updates.
```

```
class PerspectiveUpdate(PerspectiveBase):
```

```
    username: Optional[str] = Field(None, min_length=1, description="Username cannot be empty")
```

```
    layout_name: Optional[str] = Field(None, min_length=1, description="Layout name cannot be empty")
```

```
    updated_by: Optional[str] = Field(None, min_length=1, description="Updater's email cannot be empty")
```

```
    column_state: Optional[List[ColumnState]] = None
```

```
    sort_model: Optional[List[ViewSetting]] = None
```

```
    filter_model: Optional[List[ViewSetting]] = None
```

```
# Full schema for a perspective, including auto-generated fields.
```

```
class Perspective(PerspectiveBase):
```

```
    id: int
```

```
    updated_time: datetime
```

```
class Config:
```

```
    from_attributes = True
```

services/perspective.py

```
import psycopg2
```

```
from psycopg2.extras import DictCursor
```

```
from typing import List, Optional
```

```
import json
```

```
from ..models.perspective import Perspective as PerspectiveModel
```

```
from ..schemas.perspective import PerspectiveCreate, PerspectiveUpdate
```

```
from psycopg2.extensions import connection, cursor
```

```
class PerspectiveService:
```

```
    """Service class for performing CRUD operations on Perspective data using psycopg2."""
```

```
    def __init__(self, db_conn: connection, db_curr: cursor):
```

```
        self.db_conn = db_conn
```

```
        self.db_curr = db_curr
```

```

def get_all_perspectives(self) -> List[PerspectiveModel]:
    """Retrieves all perspective records from the database."""
    self.db_curr.execute("SELECT * FROM recsui.perspectives;")
    perspectives = self.db_curr.fetchall()
    return [PerspectiveModel.from_dict(p) for p in perspectives]

def get_perspective_by_id(self, perspective_id: int) -> Optional[PerspectiveModel]:
    """Retrieves a single perspective record by its ID."""
    self.db_curr.execute("SELECT * FROM recsui.perspectives WHERE id = %s;",
(perspective_id,))
    perspective = self.db_curr.fetchone()
    return PerspectiveModel.from_dict(perspective)

def get_perspective_by_username(self, username: str) -> Optional[PerspectiveModel]:
    """Retrieves a single perspective record by its username."""
    self.db_curr.execute("SELECT * FROM recsui.perspectives WHERE username = %s;",
(username,))
    perspective = self.db_curr.fetchone()
    return PerspectiveModel.from_dict(perspective)

def create_perspective(self, perspective_in: PerspectiveCreate) -> PerspectiveModel:
    """Creates a new perspective record in the database."""
    # Convert Pydantic models to JSON strings for database insertion
    column_state_json = json.dumps([cs.model_dump() for cs in
perspective_in.column_state])
    sort_model_json = json.dumps([sm.model_dump() for sm in perspective_in.sort_model])
    filter_model_json = json.dumps([fm.model_dump() for fm in perspective_in.filter_model])

    try:
        self.db_curr.execute(
            """
            INSERT INTO recsui.perspectives (username, layout_name, updated_by,
column_state, sort_model, filter_model)
            VALUES (%s, %s, %s, %s, %s, %s)
            RETURNING *;
            """,
            (
                perspective_in.username,
                perspective_in.layout_name,
                perspective_in.updated_by,
                column_state_json,
                sort_model_json,
                filter_model_json
            )

```

```

    )
)
new_perspective = self.db_curr.fetchone()
self.db_conn.commit()
return PerspectiveModel.from_dict(new_perspective)
except Exception as e:
    self.db_conn.rollback()
    raise e

```

```

def update_perspective(self, perspective_id: int, perspective_in: PerspectiveUpdate) ->
Optional[PerspectiveModel]:

```

```

    """Updates an existing perspective record."""

```

```

    # Check if the perspective exists

```

```

    perspective_to_update = self.get_perspective_by_id(perspective_id)

```

```

    if not perspective_to_update:

```

```

        return None

```

```

    # Build the update query dynamically

```

```

    update_clauses = []

```

```

    update_data = []

```

```

    # Convert Pydantic models to JSON strings for database update

```

```

    perspective_dict = perspective_in.model_dump(exclude_unset=True)

```

```

    if 'username' in perspective_dict:

```

```

        update_clauses.append("username = %s")

```

```

        update_data.append(perspective_dict['username'])

```

```

    if 'layout_name' in perspective_dict:

```

```

        update_clauses.append("layout_name = %s")

```

```

        update_data.append(perspective_dict['layout_name'])

```

```

    if 'updated_by' in perspective_dict:

```

```

        update_clauses.append("updated_by = %s")

```

```

        update_data.append(perspective_dict['updated_by'])

```

```

    if 'column_state' in perspective_dict:

```

```

        update_clauses.append("column_state = %s")

```

```

        update_data.append(json.dumps([cs.model_dump() for cs in
perspective_in.column_state]))

```

```

    if 'sort_model' in perspective_dict:

```

```

        update_clauses.append("sort_model = %s")

```

```

        update_data.append(json.dumps([sm.model_dump() for sm in
perspective_in.sort_model]))

```

```

    if 'filter_model' in perspective_dict:

```

```

        update_clauses.append("filter_model = %s")

```

```

        update_data.append(json.dumps([fm.model_dump() for fm in
perspective_in.filter_model]))

    if not update_clauses:
        return perspective_to_update # No changes to apply

    query = f"UPDATE recsui.perspectives SET {' '.join(update_clauses)} WHERE id = %s
RETURNING *;"
    update_data.append(perspective_id)

    try:
        self.db_curr.execute(query, tuple(update_data))
        updated_perspective = self.db_curr.fetchone()
        self.db_conn.commit()
        return PerspectiveModel.from_dict(updated_perspective)
    except Exception as e:
        self.db_conn.rollback()
        raise e

def update_perspective_by_username(self, username: str, perspective_in:
PerspectiveUpdate) -> Optional[PerspectiveModel]:
    """Updates an existing perspective record by its username."""
    # Check if the perspective exists
    perspective_to_update = self.get_perspective_by_username(username)
    if not perspective_to_update:
        return None

    # Build the update query dynamically
    update_clauses = []
    update_data = []

    # Convert Pydantic models to JSON strings for database update
    perspective_dict = perspective_in.model_dump(exclude_unset=True)

    if 'username' in perspective_dict:
        update_clauses.append("username = %s")
        update_data.append(perspective_dict['username'])
    if 'layout_name' in perspective_dict:
        update_clauses.append("layout_name = %s")
        update_data.append(perspective_dict['layout_name'])
    if 'updated_by' in perspective_dict:
        update_clauses.append("updated_by = %s")
        update_data.append(perspective_dict['updated_by'])

```

```

        if 'column_state' in perspective_dict:
            update_clauses.append("column_state = %s")
            update_data.append(json.dumps([cs.model_dump() for cs in
perspective_in.column_state]))
        if 'sort_model' in perspective_dict:
            update_clauses.append("sort_model = %s")
            update_data.append(json.dumps([sm.model_dump() for sm in
perspective_in.sort_model]))
        if 'filter_model' in perspective_dict:
            update_clauses.append("filter_model = %s")
            update_data.append(json.dumps([fm.model_dump() for fm in
perspective_in.filter_model]))

    if not update_clauses:
        return PerspectiveModel.from_dict(perspective_to_update) # No changes to apply

    query = f"UPDATE recsui.perspectives SET {' '.join(update_clauses)} WHERE username =
%s RETURNING *;"
    update_data.append(username)

    try:
        self.db_curr.execute(query, tuple(update_data))
        updated_perspective = self.db_curr.fetchone()
        self.db_conn.commit()
        return PerspectiveModel.from_dict(updated_perspective)
    except Exception as e:
        self.db_conn.rollback()
        raise e

def delete_perspective(self, perspective_id: int) -> bool:
    """Deletes a perspective record by its ID."""
    try:
        self.db_curr.execute("DELETE FROM recsui.perspectives WHERE id = %s RETURNING
id;", (perspective_id,))
        deleted_row = self.db_curr.fetchone()
        if deleted_row:
            self.db_conn.commit()
            return True
        else:
            self.db_conn.rollback()
            return False
    except Exception as e:

```

```
self.db_conn.rollback()  
raise e
```